BYO AI ACCOUNT BRIDGE / RESEARCH SERIES

# Security & Compliance

Threat model, credential controls, data handling and policy decision process

05

## DETAILED TECHNICAL DOSSIER

User-owned accounts / Local-first runtime / Optional cloud relay

Research cutoff: 26 October 2026

A Consumer-web adapter (permission-gated)

B Official account authorization where supported

C User-owned free APIs and locally running models

## INSIDE THIS VOLUME

**01** Executive risk finding

**02** Security objectives and assets

**03** Credential custody policy

**04** Authorization and remote tunnel hardening

**05** Privacy data-flow inventory

**06** Retention and data subject controls

**07** Provider conditions and legal classification

**08** Secure development and release process

**09** Incident response runbook

**10** Minimum external-review checklist

Research notation: [Sxx] markers link to first-party terms, official technical documentation, published standards or original project repositories. Statements labeled proposed, target, or estimate are design judgments, not observed performance.

# Executive risk finding

The biggest product risk is **permission to turn consumer UI access into a programmable service**, followed by protection of user sessions and local-to-cloud transport. The design must not assume that each customer bringing their own login removes provider restrictions. OpenAI and Anthropic impose explicit automated-access limits in their consumer terms, and Anthropic separately addresses third-party Claude Code routing. Real consumer web connectors require provider authorization before public enablement. **[S01] [S02] [S03]**

This document is an engineering and product risk analysis, **not legal advice**. Final release review needs counsel familiar with the countries and provider contracts involved, and updated terms immediately before shipment.

# Security objectives and assets

**Primary assets:** (1) provider login/browser-profile state, OAuth refresh/access grants and API keys; (2) user's prompts and model responses; (3) device private signing keys; (4) remote gateway keys; (5) routing-policy consent and paid-use limits; (6) cloud user/device metadata; (7) integrity of connector code and updates. Threat modeling should be repeated after architecture changes and documented as an evolving exercise. **[S29]**

**Adversaries:** malicious websites targeting localhost, malware on user machine, stolen laptop, leaked logs, phished dashboard account, malicious dashboard operator, compromised relay, another dashboard tenant, bad connector update, and a legitimate user attempting cross-account/over-quota routing.

### STRIDE-style attack register

| Threat | Impact | Mandatory mitigation | Validation |
|---|---|---|---|
| Website calls localhost gateway via browser | Unauthorized inference, account expenditure | Loopback-only bind, API key, CORS/Origin/Host checks, CSRF defenses | Malicious-origin browser tests **[S37]** |
| Stolen browser profile/cookie | Full consumer-account impersonation | Local isolated profiles, restricted filesystem ACLs, keychain secrets; no export | Test filesystem permissions and backup leakage **[S21]** |
| Cloud key used against different tenant's device | Cross-user prompt or billing breach | Per-request device owner check, scoped key and row-level query authorization | Cross-tenant BOLA tests **[S38]** |
| Relay impersonation or tampering | Exfiltration or altered generation | TLS/WSS, mutually bound device key, signed challenges, bounded frames | MITM simulation and replay tests **[S19]** |
| Plaintext secret in logs or crash report | Account compromise | Structured redaction, allowlist logging, canary secret scanner | Inject canary token; assert no export **[S20]** |
| Unapproved fallback to paid model | Unexpected charges and processor change | Budget=0 default, consent per route, hard router cap | Fault-injection on quota exhaustion |
| Connector malicious update | Steals all local sessions | Signed updates, pinned origin, minimal privileges, manifest permissions, rollback | Tampered-package rejection |
| Denial-of-service on remote key | Workstation resource depletion | Rate caps, pending queue, concurrency, request size/time limits | Burst/load tests |
| Prompt inclusion in analytics | Confidential data retention | Metrics only, no body logs by default, opt-in diagnostics | Audit packet and dashboard export |
| Stolen device with persistent remote trust | Remote requests still permitted | Device revoke, short-lived cloud sessions, key rotation, local startup lock | Revoke during active session |

OWASP API Security Top 10 emphasizes broken object-level authorization and authentication; WebSocket security calls for explicit authentication/authorization, origin checks, bounds, and session handling. **[S19] [S28] [S38]**

## Credential custody policy

- A user logs into a provider only on a provider-controlled page or approved native OAuth window. Do **not** collect passwords or account cookies in dashboard forms.
- Browser-session profiles are encrypted-at-rest where feasible, live in isolated OS-user directories, and never sync to provider cloud storage or the product's cloud database. Full-disk encryption should be recommended but not silently assumed.

- Official API keys and OAuth refresh material live behind platform Keychain/Credential Manager/Secret Service, or authenticated encrypted local storage when OS facilities unavailable.
- Local app displays permission provenance and a one-click disconnect; account revocation clears local references, stops queued work, and revokes OAuth provider-side if supported.
- Diagnostic export is content-free by default; all tokens must be redacted, even from error strings, command lines and child process environment dumps.
- Remote gateway keys are generated using a cryptographically secure RNG, shown once, stored as salted hashes server-side and scoped to user/device/method; user can set expiry and revoke immediately.
- Pairing key is separate from provider credentials. Revoke device does not delete the person's provider account; remove local profile only after explicit confirmation.

OWASP advises secrets inventory, rotation, least privilege and avoidance of logs; persistent Playwright authentication state is sensitive. **[S20] [S21]**

## Authorization and remote tunnel hardening

The cloud must evaluate every request against `gateway_key.owner_id == device.owner_id`, active device pairing, user remote-consent flag, allowed model aliases, spend cap and concurrent-request limit. Never trust a `device_id` supplied in a request as proof of ownership. Device keys should be created and stored locally, with public keys registered in cloud.

**Pairing protocol (recommended):** user authenticates to dashboard; dashboard creates short-lived single-use code; local daemon generates asymmetric key pair; daemon exchanges code and public key over TLS; dashboard presents owner/device identity to user; paired daemon proves possession in an authenticated challenge; store revocation marker. No unattended silent auto-pairing.

**Socket protocol:** only outbound WSS from desktop; authenticate on connect and re-auth on expiry; validate `Origin` for browser-based sockets; include heartbeat and liveness expiry; cap frame sizes and stream buffers; validate per-message schema; reject stale or replayed request IDs; cancel inflight operations after revocation. **[S19]**

**OAuth for desktop:** use authorization code + PKCE and system browser where provider allows; avoid embedded credential capture. RFC 9700 sets modern OAuth security best practices; RFC 8252 covers native applications and loopback redirect considerations. Store `state`/nonce, protect against mix-up, honor refresh-token rotation, and do not exchange consumer-session cookies for pseudo-access tokens. **[S26] [S27]**

# Privacy data-flow inventory

| Event | Data entering system | Data stored where | User disclosure |
|---|---|---|---|
| Local generation | Prompt and response | In memory; no body logs default | Sent directly from device to chosen model provider |
| Remote generation | Prompt, response, gateway ID | Relay memory transiently; cloud stores metadata only | Both cloud relay and chosen provider can process content |
| Browser A request | Prompt entered in provider UI | Provider may retain chat history under own settings | User must understand that UI history/policies apply |
| Official B request | Text and grant | OAuth credential local; provider handles request | Model, allowance and grant scope are provider controlled |
| Free C API | Prompt and free API credential | Key local; provider processes content | Free-tier training/data-use policy and region warning **[S08]** |
| Local Ollama | Prompt and response | Device memory/local service, depending on user config | No proprietary external provider required **[S13]** |
| Dashboard usage | Timestamp, route class, status, duration | Cloud short-lived metadata | Visible retention and deletion setting |

**Do not promise end-to-end encryption in remote mode** unless the app adds true payload cryptography between client and daemon with an audited key establishment protocol and the cloud cannot access decrypted message material. TLS from client to cloud plus WSS from cloud to local protects network hops, not data from the relay operator.

# Retention and data subject controls

Recommended defaults: no prompt/response retention; local event metadata 7 days; cloud identifiable event metadata 7 days, then aggregate/deidentify for at most 30 additional days; security audit events retained based on documented threat and legal need; provider cookies live until user disconnects; cloud account metadata removed on deletion subject to limited legally required hold. These are proposed settings, not legal requirements.

The app should offer: transparent export of profile/settings/usage metadata; account deletion workflow; local profile deletion and credential revoke; device disconnect; analytics opt-out; specific per-connector privacy notices; user choice to block remote relay for sensitive requests. A user cannot necessarily delete data already processed and retained by an external model provider through this product; link them to provider controls.

# Provider conditions and legal classification

| Rule source | Observed control | Product implication |
|---|---|---|
| OpenAI consumer terms | Restrict automatic/programmatic extraction and interference with service limits **[S01]** | Do not ship unauthorized ChatGPT web automation |
| Anthropic consumer terms | Restrict automatic/non-human use except permitted means **[S02]** | Explicit authorization required for consumer-Claude automation |
| Claude Code legal/compliance | Distinguishes official API and plan credentials in third-party routing contexts **[S03]** | No third-party consumer OAuth token proxy by default |
| OpenAI SIWC docs | Local OSS grants with scope, preview constraints, remote/hosted eligibility **[S04]** **[S07]** | B is narrow and conditional; verify distribution rights |
| Google Terms and Gemini API Terms | Separate general service and API use with region/data distinctions **[S08]** **[S24]** | Prefer official API with explicit terms and privacy review |
| Gemini free-tier service conditions | Free-tier data may have different use terms; billing/model constraints **[S08]** **[S40]** | Give per-route consent and retention disclosure |
| MCP specification | Protocol changes not equivalent to commercial usage rights **[S14]** | MCP wrapper grants no extra authority to provider accounts |

**Additional legal workstream:** determine whether device software, optional proxy routing and cloud dashboard count as a derivative client, automated access, reselling or sharing; validate terms in each target jurisdiction; publish privacy notice and terms of use; assess GDPR/UK GDPR, Indian DPDP, US state privacy rules and cross-border transfer requirements based on actual operations. Those laws change; this report does not assert current threshold applicability. Obtain counsel before collecting EU/UK-sensitive data or offering regulated workloads.

# Secure development and release process

**At code review:** threat-model connector changes, authorize network domains, vet npm packages and browser extensions, perform secret scanning, dependency audits, reproducible builds where practicable, signed artifacts and update channel separation. Never ship developer debugging ports exposed on LAN. Use CSP for dashboard, secure cookies, session expiry, logout invalidation, and strong account recovery.

**At QA:** test broken object-level authorization against every device and key endpoint; test websocket unauthorized messages and replay; verify prompt content absent in all logs and tracing; exercise device stolen/revoked, account lock, app reinstall, canceled stream, lost network, quotas, free/paid route toggles and browser profile deletion. Test no prompt fallbacks after explicit 'local only' setting.

**Provider permission release gate:** record authorizing document, exact endpoints/UI surface permitted, purpose, redistribution/commercial permission, account plan eligibility, quota constraints, region restrictions, provider contact and next review date. A green functional test is not enough.

## Incident response runbook

- **Detect and scope:** classify incident type (credential exposure, cross-tenant routing, unauthorized provider calls, silent paid fallback, sensitive prompt logging, malicious update).
- **Contain:** disable connector or remote relay with a server-side kill switch; revoke affected gateway/device keys; stop suspicious agent versions; preserve minimal non-content audit evidence.
- **Protect users:** force re-auth, request provider token revocation and warn about consumer-session invalidation if profile state leaked.
- **Investigate:** trace request IDs and device IDs without unnecessarily exporting prompts; identify exposure window and affected owners.
- **Notify:** follow applicable legal deadlines and contractual obligations with counsel; give actionable remediation instructions.
- **Recover:** ship signed patch; monitor, review root cause and add regression test before reconnecting provider.

## Minimum external-review checklist

- [ ] Provider-permission/legal review for every A or B connector and planned hosting mode.
- [ ] Independent penetration test of local gateway/browser-origin isolation and cloud BOLA.
- [ ] Manual audit of secrets stored in files, logs, analytics, crash reports and exports.
- [ ] Pairing/revocation replay and cross-user/device test suite passes.
- [ ] Incident response owner, reporting channel and disclosure notice are published.
- [ ] External privacy/security policy describes the exact relay plaintext visibility.
- [ ] Vulnerability intake, update signing and rollback are operational.

# Evidence & source register

Sources were checked in October 2026. Their content, access conditions, prices and quotas can change. Project READMEs are evidence of what authors claim to implement, not proof of provider permission or reliability.

**[S01] OpenAI consumer Terms of Use (effective January 2026)**
https://openai.com/policies/row-terms-of-use/

**[S02] Anthropic Consumer Terms of Service (effective October 2025)**
https://cdn.sanity.io/files/4zrzovbb/website/43c13f51a6d9755341433f79cf63c966ed9200ea.pdf

**[S03] Anthropic Claude Code - Legal and compliance**
https://code.claude.com/docs/en/legal-and-compliance

**[S04] OpenAI Sign in with ChatGPT - Local open-source plan usage**
https://developers.openai.com/siwc/token-sharing-open-source

**[S07] OpenAI Help - Using ChatGPT plans in other apps**
https://help.openai.com/en/articles/20001542-using-your-chatgpt-plan-in-other-apps-and-sites

**[S08] Google Gemini API Additional Terms (effective March 2026)**
https://ai.google.dev/gemini-api/terms

**[S13] Ollama OpenAI-compatible API**
https://docs.ollama.com/api/openai-compatibility

**[S14] MCP specification release, 28 July 2026**
https://blog.modelcontextprotocol.io/posts/2026-07-28/

**[S19] OWASP WebSocket Security Cheat Sheet**
https://cheatsheetseries.owasp.org/cheatsheets/WebSocket_Security_Cheat_Sheet.html

**[S20] OWASP Secrets Management Cheat Sheet**
https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

**[S21] Playwright authentication and browser-state guidance**
https://playwright.dev/docs/auth

**[S24] Google Terms of Service (July 2026)**
https://policies.google.com/terms?hl=en-US

**[S26] OAuth 2.0 Security Best Current Practice - RFC 9700**
https://www.rfc-editor.org/rfc/rfc9700.html

**[S27] OAuth for native applications - RFC 8252**
https://www.rfc-editor.org/rfc/rfc8252.html

**[S28] OWASP API Security Top Ten, 2023**
https://api-security.owasp.org/editions/2023/en/0x11-t10/

**[S29] OWASP Threat Modeling Cheat Sheet**
https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html

**[S37] OWASP Cross-Site Request Forgery guidance**
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

**[S38] OWASP API Broken Object Level Authorization**
https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/

**[S40] Gemini API pricing by model**
https://ai.google.dev/gemini-api/docs/pricing

Evidence boundary: this series is a design-and-research package, not a working implementation, legal opinion, measured security certification or promise of free access to any specific proprietary model.